Sumario

¿Qué es Markdown?.....7
Mi título principal.....7
Diferencias frente a HTML y otros formatos de texto.....7
1. Markdown vs. HTML.....7
2. Markdown vs. Rich Text (Word, Google Docs).....8
3. Markdown vs. Otros formatos (LaTeX, WYSIWYG, etc.).....8
En resumen.....8
Historia y origen de Markdown.....8
Creación por John Gruber y Aaron Swartz.....8
Objetivo de simplicidad y legibilidad.....9
Simplicidad al escribir.....9
Legibilidad en texto plano.....9
Conversión fácil a otros formatos.....10
Expansión y adopción.....10
Conclusión.....10
Estructura básica de un archivo Markdown (.md).....11
3.1 Convenciones de extensión.....11
3.2 Compatibilidad en distintas plataformas.....11
1. GitHub Flavored Markdown (GFM).....11
2. Plataformas de blogs y documentación.....12
3. Aplicaciones de notas y productividad.....12
4. Mensajería y colaboración.....12
5. Conversión a otros formatos.....12
3.3 Ejemplo de estructura básica de un archivo .md.....12
Proyecto Ejemplo.....13
Descripción.....13
Características principales:.....13
Enlace de interés.....13
Imagen de ejemplo.....13
Conclusión.....13
Encabezados y jerarquías de títulos en Markdown.....14
4.1 Uso de #, ##, ###, etc.....14
Encabezado 1.....14
Encabezado 2.....14
Encabezado 3.....14
Encabezado 4.....14
Encabezado 5.....14
Encabezado 6.....14
4.2 Encabezado con estilo alternativo.....14
Encabezado nivel 1.....15
Encabezado nivel 2.....15
4.3 Jerarquía de títulos en documentos.....15
4.4 Buenas prácticas para organizar contenido.....16
4.5 Ejemplo completo de estructura de documento.....16
Conclusión.....17
Párrafos y saltos de línea en Markdown.....17

5.1 Párrafos en Markdown....17
5.2 Saltos de línea dentro de un párrafo....17
5.3 Espacios y reglas de separación....18
5.4 Diferencias entre Markdown clásico y variantes....18
1. Markdown clásico (John Gruber, 2004)....18
2. GitHub Flavored Markdown (GFM)....18
3. Otras variantes (CommonMark, Markdown Extra, etc.)....19
5.5 Ejemplo práctico comparativo....19
5.6 Buenas prácticas....19
Conclusión....19
Negritas, cursivas y tachado en Markdown....20
6.1 Negritas (texto)....20
6.2 Cursivas (texto)....20
6.3 Tachado (texto)....20
6.4 Combinaciones de estilos....21
6.5 Buenas prácticas....21
6.6 Ejemplo completo....21
Ejemplo de estilos en Markdown....22
Conclusión....22
Listas ordenadas y no ordenadas en Markdown....22
7.1 Listas no ordenadas (viñetas)....22
7.2 Listas ordenadas (numeradas)....23
7.3 Sublistas y niveles de anidación....23
7.4 Listas mixtas (ordenadas + no ordenadas)....24
7.5 Buenas prácticas con listas en Markdown....25
7.6 Ejemplo completo....25
Ejemplo de listas en Markdown....26
Lista no ordenada....26
Lista ordenada....26
Lista con subniveles....26
Lista mixta....26
Conclusión....26
Enlaces e hipervínculos en Markdown....27
8.1 Sintaxis básica [texto](url)....27
8.2 Enlaces con títulos....27
8.3 Enlaces relativos....27
8.4 Enlaces internos (anclas dentro del mismo documento)....28
8.5 Diferencias entre variantes de Markdown....28
8.6 Buenas prácticas....28
8.7 Ejemplo completo....28
Ejemplo de enlaces en Markdown....29
Enlace externo....29
Enlace con título....29
Enlace relativo....29
Enlace interno (ancla en el mismo archivo)....29
URL sin formato....29
Conclusión....29
Imágenes en Markdown....29

9.1 Sintaxis básica....29
9.2 Texto alternativo (alt text)....30
9.3 Imágenes locales....30
9.4 Imágenes remotas....30
9.5 Imágenes con título....31
9.6 Imágenes clicables (enlaces + imágenes)....32
9.7 Buenas prácticas....33
9.8 Ejemplo completo....33
Ejemplo de imágenes en Markdown....34
Imagen remota....34
Imagen local....34
Imagen con título....34
Imagen clicable (enlace + imagen)....35
Conclusión....35
Citas y citas anidadas en Markdown....35
10.1 Uso de > para citas....35
10.2 Uso de >> para citas anidadas....36
10.3 Citas con otros elementos....36
Nota importante....36
10.4 Uso en documentación y notas....36
1. Documentación técnica....36
2. Notas explicativas en apuntes....37
3. Citas de texto o frases....37
4. Comentarios en discusiones....37
10.5 Buenas prácticas....37
10.6 Ejemplo completo....37
Ejemplo de citas en Markdown....38
Cita simple....38
Cita anidada....38
Cita con formato....38
Advertencia....38
Cita de autor....38
Conclusión....38
Código en línea y bloques de código en Markdown....39
11.1 Código en línea (inline)....39
11.2 Bloques de código con triple backticks....39
11.3 Especificación de lenguaje para resaltado de sintaxis....40
11.4 Alternativa con indentación (Markdown clásico)....40
11.5 Buenas prácticas....41
11.6 Ejemplo completo....41
Ejemplo de código en Markdown....41
Código en línea....41
Bloque de código simple....41
Bloque de código con resaltado....42
Bloque con SQL....42
Conclusión....42
Tablas en Markdown....42
12.1 Sintaxis básica con | y -....42

12.2 Alineación de columnas....43
12.3 Ejemplo con distintos tipos de datos....43
12.4 Tablas con formato extendido....43
12.5 Limitaciones....44
12.6 Buenas prácticas....44
12.7 Ejemplo completo....44
Ejemplo de tablas en Markdown....45
Tabla básica....45
Tabla con alineación....45
Tabla con formato y enlaces....45
Conclusión....45
Separadores y reglas horizontales en Markdown....46
13.1 Sintaxis básica....46
13.2 Variantes y estilo visual....46
13.3 Aplicaciones comunes....46
1. Dividir secciones de un documento....46
Introducción....47
Desarrollo....47
2. Resaltar un cambio de tema....47
3. Separar contenido visualmente en documentación....47
4. Notas y comentarios destacados....47
13.4 Buenas prácticas....48
13.5 Ejemplo completo....48
Documentación de Ejemplo....49
Introducción....49
Instalación....49
Uso....49
Créditos....49
Conclusión....49
Markdown extendido (GitHub Flavored Markdown - GFM)....49
14.1 Listas de tareas con checkboxes....50
Lista de tareas....50
Preparación....50
14.2 Resaltado de sintaxis extra....50
14.3 Otros extras de GFM (bonus)....51
14.4 Buenas prácticas en GFM....51
14.5 Ejemplo completo....52
Ejemplo de GFM....52
Lista de tareas....52
Bloque de código con resaltado....52
JSON con resaltado....52
Enlace automático....53
Conclusión....53
Uso de Markdown en documentación técnica....53
15.1 El rol de README.md en proyectos....53
15.2 Documentación en GitHub....54
15.3 Documentación en GitLab....54
15.4 Documentación en Bitbucket....54

15.5 Ventajas del uso de Markdown en documentación técnica....55
15.6 Buenas prácticas....55
Conclusión....55
Markdown en blogs y gestores de contenido....55
16.1 Plataformas que lo usan....56
1. Jekyll....56
2. Hugo....56
3. Gatsby....56
4. Hexo....56
5. Otros generadores populares....57
16.2 Integración con CMS....57
1. WordPress....57
2. Ghost....57
3. Netlify CMS....57
4. Headless CMS modernos....57
5. Notion y Obsidian (semi-CMS)....57
16.3 Ventajas de usar Markdown en blogs y CMS....57
16.4 Buenas prácticas....58
16.5 Ejemplo completo de post en Hugo....58
Conclusión....59
Conversión de Markdown a otros formatos....59
17.1 Pandoc: la herramienta universal....59
Instalación....59
Uso básico....59
17.2 Conversión a HTML....59
17.3 Conversión a PDF....59
17.4 Conversión a DOCX (Microsoft Word)....60
17.5 Otras herramientas de conversión....60
17.6 Ejemplo práctico con Pandoc....60
17.7 Buenas prácticas al exportar....60
Conclusión....61
Herramientas y editores para Markdown....61
18.1 Visual Studio Code (VS Code)....61
18.2 Typora....61
18.3 Obsidian....62
18.4 Otras herramientas destacadas....62
18.5 Extensiones y previsualizadores....62
18.6 Buenas prácticas al elegir editor....62
18.7 Ejemplo de flujo de trabajo....62
Conclusión....63
Buenas prácticas al escribir en Markdown....63
19.1 Legibilidad....63
19.2 Consistencia en estilos y formatos....64
19.3 Ejemplo de buenas prácticas....64
19.4 Beneficios de aplicar buenas prácticas....65
Conclusión....65
100 plataformas donde se usa Markdown....65
1. Repositorios y plataformas de desarrollo....65

2. Documentación y wikis.....66
3. Gestores de contenido (CMS estáticos y dinámicos).....66
4. Blogs y plataformas de publicación.....66
5. Aplicaciones de notas y productividad.....67
6. Foros y comunidades.....67
7. Chats y colaboración.....67
8. Gestores de proyectos.....68
9. Plataformas de e-learning y escritura académica.....68
10. Otros usos diversos.....68
11. Plataformas de chat e IA conversacional.....69

¿Qué es Markdown?

Definición y propósito

Markdown es un lenguaje de marcado ligero creado en 2004 por John Gruber, con la colaboración de Aaron Swartz.
Su objetivo principal es permitir que los textos puedan escribirse de manera sencilla, legible y clara en texto plano, pero que al mismo tiempo puedan convertirse fácilmente en documentos con formato enriquecido como HTML, PDF, Word, entre otros.
Lo que lo distingue es que la sintaxis de Markdown está pensada para que el texto, incluso sin ser procesado, sea fácil de leer. Es decir, si alguien abre un archivo .md en un editor de texto simple, podrá entender el contenido sin esfuerzo.
Ejemplo:

```
# Mi título principal

Este es un **texto importante** y este es un *texto en cursiva*.

- Punto 1
- Punto 2
- Punto 3
```

Al procesarlo, el mismo contenido se convierte en:

Mi título principal

Este es un texto importante y este es un texto en cursiva.

•Punto 1

•Punto 2

•Punto 3

Diferencias frente a HTML y otros formatos de texto

Markdown suele compararse con otros lenguajes y formatos. Veamos las principales diferencias:

1. Markdown vs. HTML

Sintaxis:
Markdown es minimalista y busca la simplicidad.
HTML es más detallado, verboso y poderoso para estructurar páginas web.
Ejemplo de título en ambos formatos:

```
# Título en Markdown
<h1>Título en HTML</h1>
```

Legibilidad:
Un archivo Markdown se entiende fácilmente como texto plano.
Un archivo HTML requiere etiquetas que interrumpen la lectura.
Alcance:
Markdown se centra en dar formato básico a documentos.
HTML sirve para crear estructuras completas de páginas web (con scripts, estilos, multimedia, etc.).

2. Markdown vs. Rich Text (Word, Google Docs)

Markdown es texto plano: puede abrirse con cualquier editor y pesa muy poco.
Rich Text (RTF, Word, Docs) son archivos binarios más pesados y dependientes de software específico.
Markdown es perfecto para programadores, documentación técnica, blogs y control de versiones (Git).
Rich Text es ideal en entornos de oficina donde se busca formato avanzado sin tocar código.

3. Markdown vs. Otros formatos (LaTeX, WYSIWYG, etc.)

LaTeX: Mucho más complejo, orientado a textos académicos, fórmulas matemáticas y tipografía avanzada. Markdown es mucho más sencillo.
Editores WYSIWYG (What You See Is What You Get, como Word o Google Docs): ofrecen edición visual, pero el contenido no siempre es tan portable o fácil de versionar como un .md.

En resumen

Markdown nació con la idea de ser rápido, ligero, portable y legible, ideal para notas, documentación, blogs y proyectos de software. A diferencia de HTML y otros formatos, no busca reemplazarlos, sino simplificar la escritura cuando no es necesario tanto nivel de detalle técnico.
¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Historia y origen de Markdown

Creación por John Gruber y Aaron Swartz

Markdown fue creado en 2004 por John Gruber, un escritor y programador estadounidense conocido por su blog tecnológico Daring Fireball. En este proyecto

también colaboró Aaron Swartz, un joven prodigio de la informática y activista por el acceso libre a la información.
John Gruber buscaba un lenguaje que le permitiera escribir artículos en su blog usando texto plano pero que pudiera convertirse rápidamente en HTML sin necesidad de escribir etiquetas largas y complicadas.
Aaron Swartz, por su parte, aportó ideas y código en Perl que facilitaron la primera implementación de Markdown, ayudando a convertir esa sintaxis sencilla en HTML automáticamente.
La primera publicación oficial donde se presentó Markdown fue en el sitio de Gruber en 2004. Allí describió su filosofía principal:

> La idea principal de Markdown es que un documento escrito en texto plano con Markdown debe ser legible como tal, sin parecer que está lleno de etiquetas o instrucciones de formato.

Objetivo de simplicidad y legibilidad

Simplicidad al escribir

No se necesita recordar etiquetas complejas como en HTML.
Basta con unos pocos símbolos fáciles de memorizar (`#`, `*`, `-`, `[]()`, etc.).
Ejemplo:

```
# Encabezado principal
Texto con **negrita** y *cursiva*.
```

Es mucho más rápido de escribir que:

```
<h1>Encabezado principal</h1>
Texto con <strong>negrita</strong> y <em>cursiva</em>.
```

Legibilidad en texto plano

Un archivo .md se puede abrir en cualquier editor básico y aun así tiene sentido. A diferencia de HTML o LaTeX, no hay ruido visual por exceso de etiquetas o comandos.
Ejemplo de lista en Markdown:

```
- Pan
- Queso
- Jamón
```

Frente a su versión en HTML:

```
<ul>
  <li>Pan</li>
  <li>Queso</li>
  <li>Jamón</li>
</ul>
```

👉 En texto plano, la lista en Markdown se entiende sin necesidad de renderizarla.

Conversión fácil a otros formatos

Con herramientas como Markdown.pl (el primer conversor en Perl creado por Gruber) o más modernas como Pandoc, un documento .md puede transformarse en HTML, PDF, DOCX o presentaciones con un solo comando.

Expansión y adopción

Aunque en 2004 fue lanzado como una herramienta para el blog de Gruber, Markdown se expandió rápidamente en la comunidad de desarrolladores gracias a su simplicidad. Con el tiempo:

•Plataformas de código abierto como GitHub, GitLab y Bitbucket adoptaron Markdown para los famosos archivos README.md.

•CMS y generadores de sitios estáticos (como Jekyll, Hugo, Hexo) lo incorporaron para crear blogs y documentación técnica.

•Herramientas de notas como Obsidian, Notion, Typora, Bear lo popularizaron en ámbitos de productividad personal.

•Hoy incluso servicios de mensajería como Slack y Discord soportan partes de la sintaxis Markdown (cursivas, negritas, citas, código).

Conclusión

La historia de Markdown muestra cómo una idea simple puede transformar la manera en que escribimos en el mundo digital:

•Fue creado en 2004 por John Gruber con la ayuda de Aaron Swartz.

•Su objetivo central fue la simplicidad y legibilidad, tanto en texto plano como en formato convertido.

•Con el tiempo, pasó de ser una herramienta de blog a convertirse en un estándar de facto para documentación, blogs, notas personales y comunicación en comunidades de software.

👉 En resumen: Markdown nació para que el texto fuera fácil de escribir, fácil de leer y fácil de convertir, y por eso hoy está presente en casi todos los rincones de la programación y la escritura digital.

¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Estructura básica de un archivo Markdown (.md)

Markdown se basa en texto plano con símbolos muy simples para indicar formato. Un archivo escrito en este lenguaje se guarda normalmente con la extensión .md y, al abrirlo, se puede leer sin necesidad de procesarlo, ya que la sintaxis es clara y cercana al lenguaje natural.

3.1 Convenciones de extensión

La extensión más utilizada para los archivos de Markdown es:

•.md: abreviación de Markdown.

Sin embargo, también existen otras extensiones que se utilizan en distintos contextos:

•.markdown: versión más explícita, menos usada por ser más larga.

•.mkd, .mkdn, .mdown: variantes históricas utilizadas en algunos proyectos.

👉 Hoy en día, la convención más aceptada y estándar es .md, especialmente en plataformas como GitHub, GitLab y Bitbucket.

Ejemplo de archivo: README.md

Es el archivo típico de introducción en repositorios de software, donde se describe qué hace el proyecto, cómo instalarlo y cómo usarlo.

3.2 Compatibilidad en distintas plataformas

Markdown se ha convertido en un estándar de facto en múltiples contextos. Aunque la base es la misma, existen pequeñas variaciones (dialectos o sabores de Markdown).

1. GitHub Flavored Markdown (GFM)

GitHub extendió el Markdown original con nuevas funcionalidades:

Listas de tareas:

```
- [x] Tarea completada
- [ ] Tarea pendiente
```

Resaltado de sintaxis en bloques de código con triple backticks:

```
```python
print("Hola, Markdown en Python")
```
```

Tablas sencillas:

```
| Nombre | Edad |
|--------|------|
| Ana    | 23   |
```

```
| Juan    | 30    |
```

Es el más usado en proyectos de software y documentación técnica.

2. Plataformas de blogs y documentación

Generadores de sitios estáticos que usan Markdown como base para escribir posts o páginas: Jekyll, Hugo, Gatsby.
Sistemas de documentación: MkDocs, Docusaurus, Sphinx.

3. Aplicaciones de notas y productividad

Obsidian, Typora, Notion, Bear usan Markdown para que las notas sean más organizadas y exportables. Evernote no soporta Markdown de manera nativa, pero se puede integrar mediante plugins.

4. Mensajería y colaboración

Slack, Discord, Telegram, WhatsApp Web aceptan algunas sintaxis de Markdown:
•**negrita** → negrita
•_cursiva_ → cursiva
•`código` → código

5. Conversión a otros formatos

Markdown es muy versátil porque puede convertirse fácilmente en:
•HTML (el destino original).
•PDF, DOCX, LaTeX mediante herramientas como Pandoc.
•Presentaciones (por ejemplo con Reveal.js o Marp).

3.3 Ejemplo de estructura básica de un archivo .md

Un archivo Markdown típico combina encabezados, párrafos, listas, enlaces e imágenes:

```
# Proyecto Ejemplo

Este es un archivo **Markdown** de ejemplo.

## Descripción
Markdown es un formato de texto ligero que se convierte fácilmente
en HTML.

### Características principales:
- Legible en texto plano
- Fácil de aprender
- Compatible con múltiples plataformas

## Enlace de interés
```

```
[Visitar GitHub](https://github.com)

## Imagen de ejemplo
![Logo Markdown](https://upload.wikimedia.org/wikipedia/commons/4/48/Markdown-mark.svg)
```

Renderizado, se vería como:

Proyecto Ejemplo

Este es un archivo Markdown de ejemplo.

Descripción

Markdown es un formato de texto ligero que se convierte fácilmente en HTML.

Características principales:

•Legible en texto plano

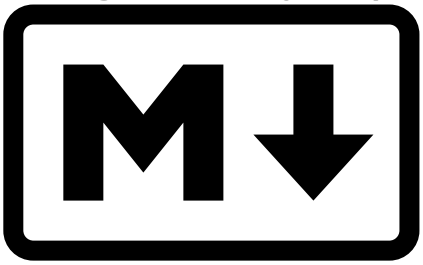
•Fácil de aprender

•Compatible con múltiples plataformas

Enlace de interés

Visitar GitHub

Imagen de ejemplo



Conclusión

Los archivos Markdown se guardan principalmente con la extensión .md, aunque existen variantes menos comunes. Son altamente compatibles en distintas plataformas, desde repositorios de software hasta gestores de blogs, aplicaciones de notas y chats.

Su estructura básica combina títulos, texto, listas y enlaces, manteniendo siempre la filosofía de legibilidad en texto plano. 👉 En definitiva, Markdown es flexible, portable y universal, lo que explica su enorme adopción.

¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Encabezados y jerarquías de títulos en Markdown

Los encabezados son la manera de estructurar y jerarquizar la información dentro de un documento en Markdown. Funcionan de forma similar a los encabezados HTML (<h1>, <h2>, <h3>, etc.), pero en Markdown se crean usando el símbolo de numeral (#).

4.1 Uso de #, ##, ###, etc.

En Markdown, la cantidad de # al inicio de una línea determina el nivel del encabezado:

```
# Encabezado de nivel 1 (<h1>)

## Encabezado de nivel 2 (<h2>)

### Encabezado de nivel 3 (<h3>)

#### Encabezado de nivel 4 (<h4>)

##### Encabezado de nivel 5 (<h5>)

###### Encabezado de nivel 6 (<h6>)
```

Renderizado:

Encabezado 1

Encabezado 2

Encabezado 3

Encabezado 4

Encabezado 5

Encabezado 6

4.2 Encabezado con estilo alternativo

Además del uso de #, Markdown soporta un estilo alternativo (menos usado, pero válido):

•Para un Encabezado 1 (H1), se escribe el texto y debajo una línea de ===.

•Para un Encabezado 2 (H2), se escribe el texto y debajo una línea de ---.

Ejemplo:

```
Encabezado nivel 1
==================

Encabezado nivel 2
------------------
```

Renderizado:

Encabezado nivel 1

Encabezado nivel 2

👉 Aunque este método funciona, hoy en día la mayoría de las plataformas recomienda usar # porque es más consistente y flexible.

4.3 Jerarquía de títulos en documentos

La jerarquía de títulos es clave para organizar el contenido de manera lógica y ordenada. Se recomienda seguir estas reglas:

•Un único # (H1) por documento: normalmente corresponde al título principal (por ejemplo: # Guía de Markdown).

•Subsecciones con ## (H2): dividen el documento en grandes apartados (ejemplo: Introducción, Instalación, Sintaxis básica, Conclusión).

•Sub-subsecciones con ### (H3): dividen a su vez cada H2 en partes más pequeñas (por ejemplo dentro de “Sintaxis básica”: Negritas, Listas, Enlaces).

•Más niveles (####, #####, ######): usarlos solo si el contenido realmente lo necesita; en la práctica pocos documentos requieren más de H3 o H4.

Ejemplo de jerarquía bien organizada (Markdown):

```
# Guía de Markdown

## Introducción
Breve descripción sobre qué es Markdown.

## Sintaxis básica
### Encabezados
Cómo usar #, ##, ###.
### Listas
Cómo usar - o 1.
### Enlaces
Cómo escribir hipervínculos.

## Conclusión
Resumen final del documento.
```

4.4 Buenas prácticas para organizar contenido

•Un solo título principal (H1): representa el tema global del archivo.

•Coherencia jerárquica: no saltes de `#` a `####`; sube o baja un nivel a la vez.

•Títulos descriptivos: frases cortas y claras, no oraciones completas.

Incorrecto: Aquí vamos a ver cómo se usan los encabezados en Markdown para estructurar documentos

Correcto: Encabezados en Markdown

•Evitar abuso de niveles: en la mayoría de casos con H1, H2 y H3 es suficiente.

•Índice automático: si la plataforma lo permite, muchas generan TOC a partir de encabezados (por ejemplo, GitHub en los README).

•Consistencia visual: si decides que los apartados principales son H2 y las subsecciones H3, respeta ese patrón en todo el documento.

4.5 Ejemplo completo de estructura de documento

```
# Tutorial de Markdown

## 1. Introducción
Breve explicación de qué es Markdown.

## 2. Sintaxis básica
### 2.1 Encabezados
Uso de #, ##, ###, etc.
### 2.2 Texto
Negritas, cursivas y tachado.
### 2.3 Listas
Ordenadas y no ordenadas.

## 3. Casos de uso
### 3.1 Documentación
Uso en proyectos de software.
### 3.2 Blogs
Plataformas que lo soportan.
### 3.3 Notas personales
Aplicaciones como Obsidian y Typora.

## 4. Conclusión
Resumen y cierre del tutorial.
```

Renderizado, este archivo sería un documento con títulos bien jerarquizados y listos para generar un índice automático.

Conclusión

Los encabezados en Markdown se crean con # al inicio de la línea (hasta seis niveles). La jerarquía de títulos ayuda a organizar el contenido en secciones claras. La mejor práctica es usar H1 una sola vez, y luego H2 y H3 para organizar subsecciones.

Una buena estructura de encabezados mejora la legibilidad, la navegación y la usabilidad del documento, sobre todo en plataformas como GitHub o GitLab.

¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Párrafos y saltos de línea en Markdown

Los párrafos y los saltos de línea son fundamentales para la legibilidad de un documento en Markdown. Aunque parecen simples, existen reglas específicas que varían según la implementación (clásico o extendido).

5.1 Párrafos en Markdown

Un párrafo en Markdown se crea simplemente escribiendo una o más líneas de texto seguidas, separadas por una línea en blanco de otros bloques.

Ejemplo:

```
Este es el primer párrafo.

Este es el segundo párrafo.
```

Renderizado:

Este es el primer párrafo.

Este es el segundo párrafo.

👉 La línea en blanco entre ambos es obligatoria para que Markdown los interprete como párrafos distintos.

5.2 Saltos de línea dentro de un párrafo

En el Markdown clásico (estándar de John Gruber): para hacer un salto de línea dentro de un mismo párrafo, hay que terminar la línea con dos o más espacios en blanco antes de presionar Enter.

Ejemplo:

```
Primera línea con salto.
Segunda línea del mismo párrafo.
```

Renderizado:

Primera línea con salto.
Segunda línea del mismo párrafo.

👉 Sin esos dos espacios, Markdown une las frases como un mismo párrafo continuo.

5.3 Espacios y reglas de separación

•Un párrafo nuevo = una línea en blanco. Si no hay línea en blanco, el texto se considera parte del mismo párrafo.

```
Esto es un párrafo.
Esto sigue siendo el mismo párrafo.
```

Renderizado: Esto es un párrafo. Esto sigue siendo el mismo párrafo.

•Salto de línea manual = dos espacios al final. Los dos espacios son invisibles en el texto plano, pero forzan un salto.

•Bloques especiales (como listas, citas o bloques de código) separan automáticamente los párrafos.

```
Este es un párrafo.

- Elemento de lista
- Otro elemento

Siguiente párrafo después de la lista.
```

Renderizado:

Este es un párrafo.

•Elemento de lista

•Otro elemento

Siguiente párrafo después de la lista.

5.4 Diferencias entre Markdown clásico y variantes

Markdown, al ser un estándar ligero, no tiene una única implementación. Diferentes plataformas han adoptado pequeñas variantes en el manejo de saltos de línea:

1. Markdown clásico (John Gruber, 2004)

•Requiere dos espacios al final de la línea para forzar un salto dentro de un párrafo.

•Si no se ponen, las frases se unen en un solo párrafo.

2. GitHub Flavored Markdown (GFM)

GFM es más permisivo: reconoce el salto de línea con solo presionar Enter, sin necesidad de los dos espacios.

Ejemplo:

```
Línea 1
Línea 2
```

En GFM, se renderiza como:

Línea 1

Línea 2

👉 Esto facilita mucho la escritura en `README.md`.

3. Otras variantes (CommonMark, Markdown Extra, etc.)

•CommonMark: busca unificar reglas. Permite Enter como salto de línea, pero conserva compatibilidad con los dos espacios.

•Markdown Extra (usado en algunos blogs y CMS como WordPress): añade opciones de estilo adicionales, pero mantiene la lógica básica.

5.5 Ejemplo práctico comparativo

```
# Ejemplo de párrafos y saltos de línea

Este es el primer párrafo.
Este texto sigue siendo parte del mismo párrafo (Markdown
clásico).

Este es un párrafo separado.

Línea con salto manual (dos espacios al final)
Nueva línea en el mismo párrafo.

Línea 1
Línea 2 (en GitHub Markdown aparece con salto automático).
```

Renderizado en Markdown clásico:

•El segundo renglón se une al primero.

•Solo el caso con dos espacios genera salto.

Renderizado en GitHub (GFM):

•Cada Enter hace un salto de línea.

•Más intuitivo para la mayoría de usuarios.

5.6 Buenas prácticas

•Usar línea en blanco para separar párrafos.

•En documentación técnica (por ejemplo en GitHub), aprovechar la compatibilidad con Enter sin espacios.

•Si se busca compatibilidad universal, preferir los dos espacios al final para forzar saltos.

•Mantener coherencia en todo el documento (no mezclar reglas de distintos sabores).

•Evitar abusar de saltos de línea para no afectar la legibilidad.

Conclusión

En Markdown, un párrafo se crea con texto seguido separado por líneas en blanco. Para un salto de línea dentro del mismo párrafo, se usan dos espacios al final de la línea (Markdown clásico) o simplemente Enter (GFM). Existen pequeñas diferencias

según la variante utilizada, pero el principio es el mismo: mantener los documentos claros, legibles y portables.

¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Negritas, cursivas y tachado en Markdown

Markdown permite aplicar formato de texto enriquecido sin perder la simplicidad del texto plano. Los estilos más básicos son negrita, cursiva y tachado. Estos se logran utilizando símbolos muy sencillos: `*`, `_` y `~`.

6.1 Negritas (texto)

La negrita se utiliza para resaltar palabras o frases importantes. En Markdown se consigue de dos formas:

Con dos asteriscos:

```
Este es un **texto en negrita**.
```

Con dos guiones bajos:

```
Este es un __texto en negrita__.
```

👉 Ambos métodos son equivalentes, aunque los asteriscos (`**`) son los más comunes en la práctica.

Renderizado: Este es un texto en negrita.

6.2 Cursivas (texto)

La cursiva se usa para énfasis suave o para destacar conceptos sin demasiada fuerza.

Con un asterisco:

```
Este es un *texto en cursiva*.
```

Con un guion bajo:

```
Este es un _texto en cursiva_.
```

👉 También son equivalentes, pero conviene ser consistente y elegir siempre `*` o `_`.

Renderizado: Este es un texto en cursiva.

6.3 Tachado (texto)

El tachado sirve para marcar texto eliminado o desactualizado. Solo funciona en variantes modernas como GitHub Flavored Markdown (GFM).

Sintaxis:

```
Este es un ~~texto tachado~~.
```

Renderizado: Este es un texto tachado.

👉 En Markdown clásico (2004) el tachado no existía. Fue agregado después en GFM y luego en otros sabores como CommonMark.

6.4 Combinaciones de estilos

Markdown permite combinar formatos dentro de una misma palabra o frase:

Negrita + cursiva

`Este es un ***texto en negrita y cursiva***.`

Renderizado: Este es un texto en negrita y cursiva.

Negrita + tachado

`Este es un **~~texto en negrita y tachado~~**.`

Renderizado: Este es un texto en negrita y tachado.

Cursiva + tachado

`Este es un *~~texto en cursiva y tachado~~*.`

Renderizado: Este es un texto en cursiva y tachado.

Negrita + cursiva + tachado

`Este es un ***~~texto con los tres estilos~~***.`

Renderizado: Este es un texto con los tres estilos.

6.5 Buenas prácticas

•Usar negrita para énfasis fuerte: títulos dentro del texto, conceptos clave, advertencias.

•Usar cursiva para énfasis suave: palabras extranjeras, citas, conceptos técnicos.

•Usar tachado con moderación: para mostrar cambios, errores o versiones desactualizadas.

•Mantener consistencia: elegir entre `*` o `_` y usarlo de forma uniforme en todo el documento.

•Evitar excesos: demasiado formato puede volver el texto difícil de leer.

6.6 Ejemplo completo

```
# Ejemplo de estilos en Markdown

Este es un *texto en cursiva*.
Este es un **texto en negrita**.
Este es un ~~texto tachado~~.

También podemos combinarlos:
- ***Cursiva y negrita al mismo tiempo***
- **~~Texto en negrita y tachado~~**
- *~~Texto en cursiva y tachado~~*
- ***~~Texto con todos los estilos a la vez~~***
```

Renderizado:

Ejemplo de estilos en Markdown

Este es un texto en cursiva.
Este es un texto en negrita.
Este es un texto tachado.
También podemos combinarlos:
•Cursiva y negrita al mismo tiempo
•Texto en negrita y tachado
•Texto en cursiva y tachado
•Texto con todos los estilos a la vez

Conclusión

Markdown usa asteriscos (*) o guiones bajos (_) para cursiva y negrita. El tachado (~~) es parte de las extensiones modernas como GFM. Los estilos pueden combinarse, dando flexibilidad sin perder simplicidad. La clave está en usarlos de manera consistente y moderada para mantener la legibilidad.
¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Listas ordenadas y no ordenadas en Markdown

Las listas son una herramienta fundamental para estructurar información en un documento Markdown. Existen dos tipos principales:
•Listas no ordenadas (viñetas): para ítems sin un orden específico.
•Listas ordenadas (numeradas): para ítems donde el orden sí importa.
Además, Markdown permite crear sublistas y niveles de anidación para representar jerarquías o estructuras más complejas.

7.1 Listas no ordenadas (viñetas)

En Markdown, una lista no ordenada puede construirse con:
•Un guion (-)
•Un asterisco (*)
•Un signo más (+)
👉 Todos son equivalentes; elegir uno solo y ser consistente es lo más recomendable.
Ejemplo:

```
- Manzana
- Banana
```

```
- Naranja
```

O también:

```
* Manzana
* Banana
* Naranja
```

Renderizado:

•Manzana

•Banana

•Naranja

7.2 Listas ordenadas (numeradas)

Para listas con orden, se utilizan números seguidos de un punto (`1.`).

Ejemplo:

```
1. Primer paso
2. Segundo paso
3. Tercer paso
```

Renderizado:

1.Primer paso

2.Segundo paso

3.Tercer paso

👉 Un detalle: en Markdown no importa qué número pongas en cada ítem, el renderizado siempre las mostrará en orden secuencial.

Ejemplo “trampa”:

```
1. Primer paso
1. Segundo paso
1. Tercer paso
```

Renderizado:

1.Primer paso

2.Segundo paso

3.Tercer paso

Esto permite que el autor no tenga que renumerar al agregar o quitar elementos.

7.3 Sublistas y niveles de anidación

Markdown permite crear sublistas dentro de listas principales usando sangrías (espacios o tabuladores). Generalmente se recomienda 4 espacios o 1 tabulación para anidar un nivel.

Ejemplo de lista no ordenada con sublista:

```
- Frutas
    - Manzana
    - Banana
```

```
    - Naranja
- Verduras
    - Lechuga
    - Tomate
```

Renderizado:

•Frutas

•Manzana

•Banana

•Naranja

•Verduras

•Lechuga

•Tomate

Ejemplo de lista ordenada con sublista:

```
1. Paso 1
    1. Subpaso A
    2. Subpaso B
2. Paso 2
    1. Subpaso C
```

Renderizado:

1.Paso 1

1.Subpaso A

2.Subpaso B

2.Paso 2

1.Subpaso C

👉 Importante: aunque pongas “1.1” o “2.1” en el texto, los motores de Markdown normalizan y enumeran los subniveles automáticamente.

7.4 Listas mixtas (ordenadas + no ordenadas)

También es posible combinar listas numeradas y con viñetas.

Ejemplo:

```
1. Compras en el supermercado
    - Pan
    - Queso
2. Tareas en casa
    - Lavar la ropa
    - Limpiar la cocina
```

Renderizado:

1.Compras en el supermercado

•Pan

•Queso

2.Tareas en casa
•Lavar la ropa
•Limpiar la cocina

7.5 Buenas prácticas con listas en Markdown

•Elegir un estilo y mantenerlo: usar siempre - o siempre * en listas no ordenadas.
•Dejar una línea en blanco antes y después de una lista, para que se renderice correctamente.
•Sangrar con 4 espacios o 1 tabulación para sublistas.
•Evitar listas demasiado profundas (más de 3 niveles dificultan la legibilidad).
•Usar listas ordenadas solo cuando el orden sea importante (instrucciones, pasos, rankings).

7.6 Ejemplo completo

```
# Ejemplo de listas en Markdown

## Lista no ordenada
- Café
- Té
- Agua

## Lista ordenada
1. Encender la computadora
2. Abrir el editor de texto
3. Escribir en Markdown

## Lista con subniveles
- Lenguajes de programación
    - Python
    - JavaScript
        - Node.js
        - React
    - Java

## Lista mixta
1. Preparar el desayuno
    - Café
    - Pan con manteca
2. Salir de casa
    - Tomar las llaves
    - Cerrar la puerta
```

Renderizado:

Ejemplo de listas en Markdown

Lista no ordenada

•Café
•Té
•Agua

Lista ordenada

1.Encender la computadora
2.Abrir el editor de texto
3.Escribir en Markdown

Lista con subniveles

•Lenguajes de programación
•Python
•JavaScript
•Node.js
•React
•Java

Lista mixta

1.Preparar el desayuno
•Café
•Pan con manteca
2.Salir de casa
•Tomar las llaves
•Cerrar la puerta

Conclusión

Markdown ofrece listas no ordenadas (con -, *, +) y listas ordenadas (con números). Se pueden crear sublistas anidadas mediante sangría (4 espacios o 1 tab). También es posible mezclar ambos tipos, manteniendo claridad en la estructura. Una lista bien organizada mejora la legibilidad, la jerarquía y la comprensión del documento.
¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Enlaces e hipervínculos en Markdown

Los enlaces son uno de los elementos más usados en Markdown, ya que permiten conectar el documento con sitios externos, otros archivos o secciones internas. Su sintaxis es simple y clara:

`[texto visible](url)`

8.1 Sintaxis básica [texto](url)

•[texto]: el texto que verá el lector.

•(url): la dirección del enlace.

Ejemplo:

```
[Ir a Google](https://www.google.com)
```

Renderizado: Ir a Google

👉 Este es el formato más utilizado y funciona en todas las variantes de Markdown (clásico, GFM, CommonMark).

8.2 Enlaces con títulos

Markdown permite añadir un título opcional al enlace, que se muestra como un tooltip (cuando el usuario pasa el mouse por encima).

Sintaxis:

```
[texto](url "Título opcional")
```

Ejemplo:

```
[Ir a GitHub](https://github.com "Plataforma de desarrollo colaborativo")
```

Renderizado: Ir a GitHub

👉 El título es útil para dar contexto adicional, pero no es obligatorio.

8.3 Enlaces relativos

En documentos Markdown dentro de proyectos (por ejemplo en GitHub), es común usar enlaces relativos a otros archivos o carpetas.

Ejemplo de enlace a un archivo en la misma carpeta:

```
[Leer introducción](introduccion.md)
```

Ejemplo de enlace a un archivo en otra carpeta:

```
[Ir al capítulo 2](docs/capitulo2.md)
```

Ejemplo de enlace a la carpeta principal:

```
[Volver al índice](../README.md)
```

👉 Muy útil en documentación técnica: permite navegar dentro del repositorio sin URLs absolutas.

8.4 Enlaces internos (anclas dentro del mismo documento)

Markdown también soporta enlaces a secciones dentro del mismo archivo, usando los identificadores de encabezados.

Ejemplo: si tenemos un título:

```
## Instalación
```

Podemos enlazarlo con:

```
[Ir a Instalación](#instalación)
```

👉 Regla general: los encabezados se convierten en identificadores en minúsculas, con espacios reemplazados por guiones (-). Algunos motores eliminan acentos en el ID.

Renderizado (ejemplo): Ir a Instalación

8.5 Diferencias entre variantes de Markdown

•Markdown clásico (2004): solo soporta [texto](url) básico.

•GitHub Flavored Markdown (GFM): soporta enlaces relativos y anclas internas automáticamente; interpreta rutas de repositorios como enlaces navegables.

•CommonMark: unifica reglas y extiende compatibilidad.

8.6 Buenas prácticas

•Usar texto descriptivo en lugar de genérico (por ejemplo, mejor “Documentación oficial de Python” que “Haz clic aquí”).

•Preferir enlaces relativos en repositorios para facilitar la portabilidad.

•Usar títulos opcionales solo cuando agreguen valor, no como duplicado del texto.

•Mantener consistencia: todos absolutos o todos relativos dentro de un proyecto.

•Evitar URLs crudas demasiado largas en el texto: mejor [texto](url) que pegar directamente la dirección.

8.7 Ejemplo completo

```
# Ejemplo de enlaces en Markdown

## Enlace externo
[Ir a Google](https://www.google.com)

## Enlace con título
[Ir a GitHub](https://github.com "Plataforma de desarrollo colaborativo")

## Enlace relativo
[Leer capítulo 2](docs/capitulo2.md)
```

```
## Enlace interno (ancla en el mismo archivo)
[Ir a la sección de Conclusión](#conclusión)

## URL sin formato
https://www.wikipedia.org
```

Renderizado:

Ejemplo de enlaces en Markdown

Enlace externo

Ir a Google

Enlace con título

Ir a GitHub

Enlace relativo

Leer capítulo 2

Enlace interno (ancla en el mismo archivo)

Ir a la sección de Conclusión

URL sin formato

https://www.wikipedia.org

Conclusión

Los enlaces en Markdown se crean con la sintaxis [texto](url). Se pueden enriquecer con títulos opcionales, enlaces relativos y anclas internas. Cada variante de Markdown agrega mejoras, siendo GFM la más flexible. Los enlaces descriptivos mejoran la accesibilidad, la claridad y la experiencia del usuario.

¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Imágenes en Markdown

Markdown permite insertar imágenes de forma muy sencilla, reutilizando casi la misma sintaxis de los enlaces, pero con un signo de exclamación (!) delante.

9.1 Sintaxis básica

La forma general es:

```
![texto alternativo](ruta)
```

•!: indica que es una imagen.

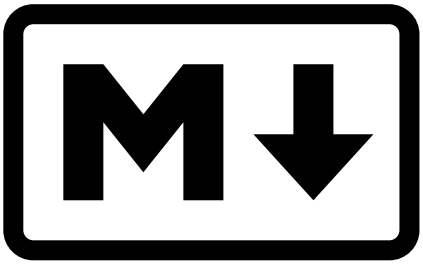
•[texto alternativo]: descripción de la imagen (importante para accesibilidad y cuando la imagen no se carga).

•(ruta): la URL o la ruta al archivo de la imagen.

Ejemplo:

```
![Logo de Markdown](https://upload.wikimedia.org/wikipedia/commons/4/48/Markdown-mark.svg)
```

Renderizado:



9.2 Texto alternativo (alt text)

El texto alternativo tiene dos propósitos principales:

•Accesibilidad: lectores de pantalla lo leen en lugar de la imagen.

•Fallback: aparece si la imagen no se puede cargar.

Ejemplo con error de URL:

```
![Imagen de ejemplo](https://ruta-no-existe.com/ejemplo.png)
```

Renderizado (fallido):

👉 El usuario verá el texto “Imagen de ejemplo” en lugar de la imagen.

9.3 Imágenes locales

En proyectos Markdown (por ejemplo, repositorios en GitHub), es común enlazar imágenes almacenadas en el mismo directorio o carpetas internas.

Ejemplo (imagen en la misma carpeta):

```
![Diagrama del sistema](diagrama.png)
```

Ejemplo (imagen en carpeta /img):

```
![Captura de pantalla](img/captura.png)
```

Ventajas:

•Se mantienen dentro del proyecto.

•Funcionan incluso sin conexión a internet.

Desventajas:

•Aumentan el tamaño del repositorio.

•Si se mueve la carpeta, hay que actualizar las rutas.

9.4 Imágenes remotas

También se pueden usar imágenes alojadas en internet mediante una URL absoluta.

Ejemplo:

```
![Logo de Python](https://www.python.org/static/community_logos/python-logo.png)
```

Renderizado:

Ventajas:

•No aumentan el tamaño del proyecto.

•No hace falta incluir archivos adicionales.

Desventajas:

•Dependen de que la URL siga funcionando.

•Necesitan conexión a internet.

9.5 Imágenes con título

Al igual que los enlaces, las imágenes pueden tener un título opcional (tooltip) entre comillas:

```
![Logo de GitHub](https://github.githubassets.com/images/modules/logos_page/GitHub-Mark.png "Logo oficial de GitHub")
```

Renderizado:

9.6 Imágenes clicables (enlaces + imágenes)

Podemos anidar una imagen dentro de un enlace:

```
[![Logo de Wikipedia](https://upload.wikimedia.org/wikipedia/commons/6/63/Wikipedia-logo.png)](https://www.wikipedia.org)
```

Renderizado (al hacer clic abre Wikipedia):

9.7 Buenas prácticas

•Usar siempre texto alternativo descriptivo, no imagen1.png.

•Preferir imágenes locales para proyectos que deben funcionar offline.

•Preferir imágenes remotas para ejemplos rápidos o cuando no controlamos el almacenamiento.

•Evitar imágenes demasiado pesadas: optimizar en formato PNG o JPG comprimido.

•Mantener organizadas las imágenes en una carpeta (/img, /assets, etc.).

•Usar títulos ("texto") cuando agreguen contexto útil.

9.8 Ejemplo completo

```
# Ejemplo de imágenes en Markdown

## Imagen remota
![Logo de Markdown](https://upload.wikimedia.org/wikipedia/commons/4/48/Markdown-mark.svg)

## Imagen local
![Diagrama del sistema](img/diagrama.png)

## Imagen con título
![Logo de GitHub](https://github.githubassets.com/images/modules/logos_page/GitHub-Mark.png "Logo oficial de GitHub")

## Imagen clicable (enlace + imagen)
[![Logo de Wikipedia](https://upload.wikimedia.org/wikipedia/commons/6/63/Wikipedia-logo.png)](https://www.wikipedia.org)
```

Renderizado:

Ejemplo de imágenes en Markdown

Imagen remota

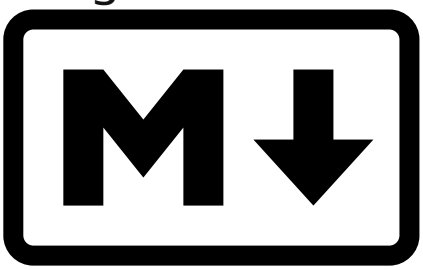


Imagen local

Imagen con título

Imagen clicable (enlace + imagen)

Conclusión

La sintaxis básica de imágenes es ![texto alternativo](ruta). Se pueden usar imágenes locales (archivos dentro del proyecto) o imágenes remotas (URLs). Markdown también permite añadir títulos y convertir imágenes en enlaces. La clave es mantener un uso organizado, accesible y eficiente de los recursos gráficos.

¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Citas y citas anidadas en Markdown

Las citas en Markdown permiten resaltar fragmentos de texto, comentarios, referencias o notas, utilizando un estilo diferenciado. Se inspiran en el formato de las citas de correo electrónico, donde se usa el símbolo > al inicio de las líneas citadas.

10.1 Uso de > para citas

La sintaxis básica para crear una cita es anteponer el símbolo > al inicio de la línea.

Ejemplo simple:

```
> Esta es una cita en Markdown.
```

Renderizado:

Esta es una cita en Markdown.

👉 Se puede aplicar a una sola línea o a párrafos completos.

Ejemplo con párrafo largo:

```
> Markdown es un lenguaje de marcado ligero creado por John Gruber
en 2004.
> Su objetivo principal es que el texto sea legible como texto
plano, pero pueda convertirse fácilmente a HTML.
```

Renderizado:

Markdown es un lenguaje de marcado ligero creado por John Gruber en 2004. Su objetivo principal es que el texto sea legible como texto plano, pero pueda convertirse fácilmente a HTML.

10.2 Uso de >> para citas anidadas

Se pueden crear citas dentro de citas (anidadas) agregando más símbolos > en cada nivel.

Ejemplo:

```
> Esta es una cita principal.
>> Esta es una cita dentro de otra.
>>> Incluso podemos anidar un tercer nivel.
```

Renderizado:

> Esta es una cita principal.
>> Esta es una cita dentro de otra.
>>> Incluso podemos anidar un tercer nivel.

👉 Aunque Markdown permite múltiples niveles de anidación, en la práctica rara vez se usan más de 2 porque pierde legibilidad.

10.3 Citas con otros elementos

Dentro de una cita también se pueden incluir texto con formato, listas, imágenes o enlaces.

Ejemplo:

```
> ### Nota importante
> - Este texto está en **negrita**.
> - También puede tener *cursiva*.
> - Y un [enlace](https://www.markdownguide.org).
```

Renderizado:

> ### Nota importante
> - Este texto está en **negrita**.
> - También puede tener *cursiva*.
> - Y un enlace.

👉 Esto es muy útil para documentar instrucciones o advertencias en proyectos.

10.4 Uso en documentación y notas

Las citas en Markdown tienen múltiples usos:

1. Documentación técnica

Para resaltar advertencias o recordatorios.

Ejemplo:

```
> **Atención**: Antes de instalar, asegurate de tener Python 3.10
o superior.
```

Renderizado:

> Atención: Antes de instalar, asegurate de tener Python 3.10 o superior.

2. Notas explicativas en apuntes

Para agregar comentarios, ejemplos o recordatorios en un documento personal.

```
> Nota: Esta fórmula se usa solo en casos especiales.
```

Renderizado:

Nota: Esta fórmula se usa solo en casos especiales.

3. Citas de texto o frases

Para resaltar frases de autores o fuentes externas.

```
> El buen diseño es obvio. El gran diseño es transparente.  Joe Sparano
```

Renderizado:

El buen diseño es obvio. El gran diseño es transparente. Joe Sparano

4. Comentarios en discusiones

En GitHub, foros o chats que usan Markdown, las citas sirven para responder a un comentario anterior.

Ejemplo en GitHub Issues:

```
> ¿Podemos usar esta función en producción?
Sí, ya está probada en staging.
```

Renderizado:

¿Podemos usar esta función en producción?

Sí, ya está probada en staging.

10.5 Buenas prácticas

•Usar citas para resaltar información, no para todo el texto.

•En documentación, aprovecharlas para advertencias, notas o recordatorios.

•No abusar de los niveles de anidación: máximo 2 para mantener claridad.

•Incluir texto alternativo claro en caso de citas con enlaces o imágenes.

•Mantener coherencia en el estilo (por ejemplo: siempre usar Nota:, Importante:, Ejemplo: dentro de una cita).

10.6 Ejemplo completo

```
# Ejemplo de citas en Markdown
```

```
## Cita simple
> Markdown es fácil de aprender.

## Cita anidada
> Primera idea.
>> Segunda idea dentro de la cita.

## Cita con formato
> ### Advertencia
> - Este comando borra todos los datos.
> - úsalo con **precaución**.

## Cita de autor
> La simplicidad es la máxima sofisticación.  Leonardo da Vinci
```

Renderizado:

Ejemplo de citas en Markdown

Cita simple

Markdown es fácil de aprender.

Cita anidada

Primera idea.

> > Segunda idea dentro de la cita.

Cita con formato

> Advertencia
>
> - Este comando borra todos los datos.
> - úsalo con precaución.

Cita de autor

La simplicidad es la máxima sofisticación. Leonardo da Vinci

Conclusión

Las citas en Markdown se crean con `>`, y se pueden anidar con `>>`, `>>>`, etc. Sirven para resaltar texto, agregar notas, incluir advertencias o citar a otros autores. Se pueden combinar con otros elementos (listas, enlaces, títulos). Bien usadas, mejoran la legibilidad y la organización de la documentación.

¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Código en línea y bloques de código en Markdown

Markdown incluye una sintaxis especial para mostrar fragmentos de código o comandos dentro de un texto, manteniendo el formato monoespaciado. Esto es fundamental en documentación técnica, tutoriales y apuntes de programación.

Existen dos formas principales:

•Código en línea (inline): para fragmentos pequeños.

•Bloques de código: para fragmentos grandes o múltiples líneas.

11.1 Código en línea (inline)

Se escribe encerrando el fragmento de código entre backticks (`), también llamados comillas invertidas.

Sintaxis:

`código`

Ejemplo:

`El comando `ls -l` muestra los archivos en formato detallado.`

Renderizado: El comando `ls -l` muestra los archivos en formato detallado.

👉 Usos típicos: resaltar nombres de funciones, variables o comandos; mostrar ejemplos cortos dentro de un párrafo.

11.2 Bloques de código con triple backticks

Cuando se necesita mostrar varias líneas de código, se utilizan tres backticks consecutivos (```) para abrir y cerrar el bloque.

Sintaxis:

```
línea 1 de código
línea 2 de código
línea 3 de código
```

Ejemplo:

```
def hola():
    print("Hola, Markdown!")
```

Renderizado:

```
def hola():
    print("Hola, Markdown!")
```

👉 Este método mantiene los saltos de línea, sangrías y formato del código.

11.3 Especificación de lenguaje para resaltado de sintaxis

Muchas plataformas (como GitHub, GitLab, Bitbucket, Obsidian, etc.) soportan resaltado de sintaxis. Para activarlo, se coloca el nombre del lenguaje justo después de los tres backticks de apertura.

Ejemplo en Python:

```
```python
def suma(a, b):
    return a + b

print(suma(2, 3))
```
```

Renderizado con resaltado:

```python
def suma(a, b):
    return a + b

print(suma(2, 3))
```

Ejemplo en JavaScript:

```
```javascript
function saludar() {
    console.log("Hola, Markdown!");
}
saludar();
```
```

Renderizado con resaltado:

```javascript
function saludar() {
    console.log("Hola, Markdown!");
}
saludar();
```

👉 El resaltado varía según el visor, pero generalmente es compatible con Python, JavaScript, Java, C/C++, C#, HTML, CSS, JSON, YAML, entre otros.

11.4 Alternativa con indentación (Markdown clásico)

En el Markdown original, antes de popularizarse los backticks triples, los bloques de código se podían crear usando 4 espacios o 1 tabulación al inicio de cada línea.

Ejemplo:

```
    SELECT * FROM usuarios WHERE activo = 1;
```

Renderizado:

```
SELECT * FROM usuarios WHERE activo = 1;
```

👉 Hoy en día, esta forma casi no se usa porque los backticks triples son más claros y permiten especificar lenguaje.

11.5 Buenas prácticas

•Usar inline solo para fragmentos cortos o nombres de comandos.

•Usar bloques con ``` para fragmentos grandes o ejemplos de código.

•Siempre que sea posible, especificar el lenguaje para mejorar la legibilidad.

•No abusar del resaltado: el código debe estar ahí para aclarar, no para saturar.

•Mantener consistencia: elegir entre ``` y la indentación clásica, pero no mezclar estilos en un mismo documento.

11.6 Ejemplo completo

```
# Ejemplo de código en Markdown

## Código en línea
Para crear un entorno virtual en Python se usa: `python -m venv
env`.

## Bloque de código simple
```
echo "Hola, Mundo!"
```

## Bloque de código con resaltado
```javascript
function multiplicar(a, b) {
   return a * b;
}
console.log(multiplicar(4, 5));
```

Bloque con SQL
```sql
SELECT nombre, edad
FROM usuarios
WHERE activo = 1;
```
```

Renderizado:

Ejemplo de código en Markdown

Código en línea

Para crear un entorno virtual en Python se usa: python -m venv env.

Bloque de código simple

```
echo "Hola, Mundo!"
```

Bloque de código con resaltado

```
function multiplicar(a, b) {
    return a * b;
}
console.log(multiplicar(4, 5));
```

Bloque con SQL

```
SELECT nombre, edad
FROM usuarios
WHERE activo = 1;
```

Conclusión

El código en línea se crea con backticks simples (`). Los bloques de código se crean con triple backticks (```) y permiten mantener saltos de línea. Se puede añadir el nombre del lenguaje para activar el resaltado de sintaxis en plataformas compatibles. Los backticks triples son hoy el estándar recomendado por su claridad y flexibilidad.

¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Tablas en Markdown

Las tablas en Markdown permiten organizar información en filas y columnas de manera sencilla. Son muy útiles en documentación técnica, comparaciones, resúmenes y presentación de datos estructurados.

👉 Fueron añadidas como extensión en variantes como GitHub Flavored Markdown (GFM), ya que el Markdown clásico (2004) no incluía soporte para tablas.

12.1 Sintaxis básica con | y -

La estructura mínima de una tabla es:

```
| Columna 1 | Columna 2 | Columna 3 |
|-----------|-----------|-----------|
| Fila 1A   | Fila 1B   | Fila 1C   |
| Fila 2A   | Fila 2B   | Fila 2C   |
```

Renderizado:

Columna 1	Columna 2	Columna 3
Fila 1A	Fila 1B	Fila 1C
Fila 2A	Fila 2B	Fila 2C

👉 Explicación:

•La primera fila define los encabezados.

•La segunda fila (con ---) separa encabezados del contenido.

•Cada fila posterior representa datos.

•Los | (pipes) delimitan las columnas.

12.2 Alineación de columnas

Se pueden alinear los textos de las columnas con : en la fila de separación.

Alineación a la izquierda (por defecto):

```
| Izquierda | Centro | Derecha |
|:----------|:------:|---------:|
| texto 1   | texto 2| texto 3  |
```

Renderizado:

Izquierda	Centro	Derecha
texto 1	texto 2	texto 3

👉 Reglas:

•:--- alineación a la izquierda.

•:---: alineación al centro.

•---: alineación a la derecha.

12.3 Ejemplo con distintos tipos de datos

```
| Nombre      | Edad | Ciudad          |
|:-----------|:---:|--------------:|
| Ana         | 23   | Córdoba         |
| Juan        | 30   | Buenos Aires    |
| Valentina   | 19   | Mendoza         |
```

Renderizado:

Nombre	Edad	Ciudad
Ana	23	Córdoba
Juan	30	Buenos Aires
Valentina	19	Mendoza

12.4 Tablas con formato extendido

Las tablas también admiten:

Celdas vacías:

```
| Producto    | Precio | Stock |
|------------|--------|-------|
| Teclado     | $15000 |  25   |
| Mouse       | $9000  |       |
```

Renderizado:

Producto	Precio	Stock
Teclado	$15000	25
Mouse	$9000	

Formato dentro de las celdas:

```
| Lenguaje    | Estado       |
|------------|-------------|
| **Python** | Popular      |
```

```
| *Rust*      | Emergente   |
| ~~Perl~~    | En desuso   |
```

Renderizado:

Lenguaje	Estado
Python	Popular
Rust	Emergente
Perl	En desuso

Enlaces e imágenes:

```
| Recurso | Enlace |
|---------|--------|
| GitHub  | [Visitar](https://github.com) |
| Logo    | ![Markdown](https://upload.wikimedia.org/wikipedia/commons/4/48/Markdown-mark.svg) |
```

Renderizado:

Recurso	Enlace
GitHub	Visitar
Logo	

12.5 Limitaciones

•No se soportan columnas combinadas (colspan/rowspan) como en HTML.

•El formato depende del visor: algunos renderizadores muestran bordes, otros solo alineación.

•Tablas muy anchas pueden ser difíciles de leer en móviles.

12.6 Buenas prácticas

•Mantener alineación coherente en columnas (por ejemplo, números a la derecha, texto a la izquierda).

•Usar espacios consistentes para mejorar legibilidad en el archivo .md.

•Evitar tablas demasiado grandes; considerar dividir en varias más simples.

•Usar tablas solo para datos tabulares, no para maquetar todo el documento.

•Cuando se comparta en GitHub, usar alineaciones claras para facilitar lectura directa en texto plano.

12.7 Ejemplo completo

```
# Ejemplo de tablas en Markdown

## Tabla básica
| Columna 1 | Columna 2 |
|-----------|-----------|
| Dato A    | Dato B    |
| Dato C    | Dato D    |
```

```
## Tabla con alineación
| Producto | Precio | Stock |
|:---------|-------:|:-----:|
| Teclado  | $15000 |  25   |
| Mouse    | $9000  |   0   |
| Monitor  | $85000 |  10   |

## Tabla con formato y enlaces
| Lenguaje   | Info |
|------------|------|
| **Python** | [Web oficial](https://www.python.org) |
| *JavaScript* |
[Documentación](https://developer.mozilla.org/es/docs/Web/JavaScri
pt) |
```

Renderizado:

Ejemplo de tablas en Markdown

Tabla básica

Columna 1	Columna 2
Dato A	Dato B
Dato C	Dato D

Tabla con alineación

Producto	Precio	Stock
Teclado	$15000	25
Mouse	$9000	0
Monitor	$85000	10

Tabla con formato y enlaces

Lenguaje	Info
Python	Web oficial
JavaScript	Documentación

Conclusión

Las tablas en Markdown se crean con pipes (`|`) y guiones (`-`). La alineación de columnas se logra con `:---` (izquierda), `:---:` (centro) y `---:` (derecha). Admiten formato enriquecido dentro de las celdas (negrita, cursiva, enlaces, imágenes). Son simples pero limitadas frente a las tablas HTML; ideales para documentación técnica clara y ligera.

¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Separadores y reglas horizontales en Markdown

Un separador o regla horizontal en Markdown se utiliza para dividir secciones dentro de un documento, proporcionando una pausa visual entre contenidos. Es el equivalente al elemento <hr> en HTML.

13.1 Sintaxis básica

Existen varias formas de crear un separador en Markdown. Todas generan el mismo resultado:

•Tres o más guiones (---)
•Tres o más asteriscos (***)
•Tres o más guiones bajos (___)

Ejemplos en Markdown:

```
- - -

***

___
```

Renderizado:

👉 Regla: deben usarse al menos tres caracteres y colocarse en una línea propia, sin texto alrededor.

13.2 Variantes y estilo visual

Aunque las tres formas (---, ***, ___) son equivalentes, algunos editores aplican diferencias de estilo:

•---: línea recta simple.
•***: algunos renderizadores la muestran con un estilo punteado o más marcado.
•___: suele renderizarse igual que ---.
👉 En la práctica, el más utilizado y estándar es ---.

13.3 Aplicaciones comunes

1. Dividir secciones de un documento

Ejemplo:

```
# Introducción
```

```
Este es el inicio del documento.

---

# Desarrollo
Aquí empieza una nueva sección.
```

Renderizado:

Introducción

Este es el inicio del documento.

Desarrollo

Aquí empieza una nueva sección.

2. Resaltar un cambio de tema

En tutoriales o apuntes, se puede usar una línea horizontal para marcar el paso de un tema a otro.

```
## Tema 1: Variables
Explicación de variables en Python.

---

## Tema 2: Condicionales
Explicación de if/else en Python.
```

3. Separar contenido visualmente en documentación

Por ejemplo, en un README.md para dividir instalación, uso y créditos.

```
# Mi Proyecto

## Instalación
Instrucciones de instalación aquí.

---

## Uso
Ejemplo de ejecución del programa.
```

4. Notas y comentarios destacados

Combinando separadores con citas (>) se pueden crear bloques visuales más claros.

```
---

> **Nota:** Este comando puede ser peligroso en producción.
```

```
---
```

Renderizado:

Nota: Este comando puede ser peligroso en producción.

13.4 Buenas prácticas

•Usar separadores solo cuando se justifique: demasiados pueden ensuciar el documento.
•En documentación técnica, emplearlos para marcar transiciones claras entre secciones.
•En notas personales, ayudan a diferenciar apuntes o comentarios.
•Elegir un estilo (---, *** o ___) y ser consistente.
•No confundir --- (separador) con - (lista no ordenada).
👉 Consejo: si al escribir --- el editor crea un título H2 en vez de un separador, asegurate de que la línea no tenga texto arriba pegado.

13.5 Ejemplo completo

```
# Documentación de Ejemplo

## Introducción
Markdown es un lenguaje de marcado ligero y fácil de usar.

---

## Instalación
1. Descargar el proyecto.
2. Ejecutar el comando `npm install`.

***

## Uso
```python
print("Hola, Markdown!")
```

Créditos

Autor: Juan Pérez
```

Renderizado:

Documentación de Ejemplo

Introducción

Markdown es un lenguaje de marcado ligero y fácil de usar.

Instalación

1. Descargar el proyecto.
2. Ejecutar el comando npm install.

Uso

```
print("Hola, Markdown!")
```

Créditos

Autor: Juan Pérez

Conclusión

Los separadores en Markdown se crean con ---, *** o ___. Sirven para dividir secciones, resaltar cambios de tema o separar contenido. Son simples, portables y mejoran la organización visual del documento. El más usado y recomendado es ---, aunque los otros son igualmente válidos.

¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Markdown extendido (GitHub Flavored Markdown - GFM)

El GitHub Flavored Markdown (GFM) es una extensión del Markdown clásico creada por GitHub para mejorar la documentación técnica dentro de los repositorios.

Incluye soporte adicional que no existía en la versión original de John Gruber (2004). Muchas de estas características hoy son estándar en plataformas como GitHub, GitLab, Bitbucket, Obsidian y VS Code.

Entre las extensiones más importantes están:

•Listas de tareas con checkboxes.

•Resaltado de sintaxis extendido.

•Enlaces automáticos, tablas y referencias (vistos en otros temas).

14.1 Listas de tareas con checkboxes

Las listas de tareas son una mejora de las listas no ordenadas. Cada ítem puede mostrarse como una casilla de verificación (checkbox), marcada o no marcada.

Sintaxis

```
- [ ] Tarea pendiente
- [x] Tarea completada
```

Renderizado

•[ ] Tarea pendiente

•[x] Tarea completada

👉 Notas:

•[ ] casilla vacía (pendiente).

•[x] casilla marcada (completada).

•Muy usadas en issues y proyectos de GitHub para llevar control visual.

Ejemplo práctico

```
# Lista de tareas

## Preparación
- [x] Instalar dependencias
- [ ] Configurar base de datos
- [ ] Probar la aplicación
```

Renderizado:

Lista de tareas

Preparación

•[x] Instalar dependencias

•[ ] Configurar base de datos

•[ ] Probar la aplicación

👉 En GitHub, las casillas pueden ser interactivas desde la interfaz.

14.2 Resaltado de sintaxis extra

Otra mejora de GFM es el soporte para resaltado en bloques de código al especificar el lenguaje tras los backticks.

Ejemplo en Python

```
```python
def saludo():
    print("Hola desde Markdown GFM")
```
```

Renderizado con resaltado:

```
def saludo():
    print("Hola desde Markdown GFM")
```

Ejemplo en JavaScript

```javascript
function sumar(a, b) {
    return a + b;
}
console.log(sumar(3, 4));
```

Renderizado:

```
function sumar(a, b) {
    return a + b;
}
console.log(sumar(3, 4));
```

Ejemplo en JSON

```json
{
  "nombre": "Ana",
  "edad": 25,
  "activo": true
}
```

Renderizado:

```
{
  "nombre": "Ana",
  "edad": 25,
  "activo": true
}
```

Lenguajes soportados

GitHub soporta más de 200 lenguajes (Python, JavaScript, Java, C/C++, C#, PHP, Go, Ruby, HTML, CSS, JSON, YAML, SQL, Bash, Dockerfile, Markdown, entre otros).

14.3 Otros extras de GFM (bonus)

•Enlaces automáticos: escribir una URL la vuelve clicable.

`https://www.github.com`

Renderizado: https://www.github.com

•Tablas mejoradas: compatibles con alineación y formato dentro de celdas (ver Tema 12).

•Emojis: códigos como :smile: que GitHub muestra como 😄.

14.4 Buenas prácticas en GFM

•Usar listas de tareas para organizar pendientes en proyectos y documentación.

•Especificar el lenguaje en los bloques de código para aprovechar el resaltado.

•Mantener consistencia en el estilo de listas y tablas para mayor legibilidad.

•Evitar funciones exclusivas de GFM si el documento circulará fuera de GitHub (emojis o checkboxes pueden no funcionar en otros visores).

14.5 Ejemplo completo

```
# Ejemplo de GFM

## Lista de tareas
- [x] Escribir introducción
- [ ] Completar ejemplos
- [ ] Revisar ortografía

## Bloque de código con resaltado
```bash
echo "Hola desde GitHub Flavored Markdown"
```

JSON con resaltado
```json
{
  "usuario": "juanperez",
  "activo": true,
  "rol": "admin"
}
```

Enlace automático
https://www.github.com
```

Renderizado:

Ejemplo de GFM

Lista de tareas

•[x] Escribir introducción

•[ ] Completar ejemplos

•[ ] Revisar ortografía

Bloque de código con resaltado

```
echo "Hola desde GitHub Flavored Markdown"
```

JSON con resaltado

```
{
  "usuario": "juanperez",
  "activo": true,
  "rol": "admin"
}
```

Enlace automático

https://www.github.com

Conclusión

GitHub Flavored Markdown (GFM) amplía el Markdown clásico con listas de tareas, resaltado extendido y extras como enlaces automáticos, tablas mejoradas y emojis. Es el estándar de facto en proyectos de software por combinar simplicidad con potencia para documentación técnica.

¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Uso de Markdown en documentación técnica

Markdown se ha convertido en el formato estándar para la documentación técnica gracias a su simplicidad, legibilidad en texto plano y compatibilidad con plataformas de desarrollo colaborativo.

Hoy en día, prácticamente todos los repositorios de software utilizan archivos .md para describir proyectos, documentar APIs y guiar a otros desarrolladores.

15.1 El rol de README.md en proyectos

El archivo README.md es el primer punto de contacto entre un proyecto y sus usuarios o colaboradores. Se ubica en el directorio raíz de un repositorio y suele aparecer automáticamente al abrirlo en plataformas como GitHub, GitLab o Bitbucket.

Contenido típico de un README.md

•Título y descripción del proyecto

•Instalación

•Uso

•Ejemplos de código

•Capturas de pantalla o imágenes

•Licencia

Ejemplo de README.md típico

```
# Gestor de Notas

Aplicación ligera para crear y organizar notas.
```

```
## Instalación
```bash
git clone https://github.com/usuario/gestor-notas.git
cd gestor-notas
npm install
```

Uso
```bash
npm start
```

Abrir en el navegador: http://localhost:3000

Características
- Crear notas
- Editar notas
- Exportar notas en PDF

Captura de pantalla
![Pantalla principal](img/captura.png)

Licencia
MIT License
```

15.2 Documentación en GitHub

GitHub utiliza Markdown como formato nativo para la documentación:

•Archivos clave: README.md, CONTRIBUTING.md, CHANGELOG.md, LICENSE.

•Wiki: wiki basado en Markdown para manuales extensos.

•Issues y Pull Requests: soportan listas de tareas, enlaces y bloques de código.

•Usa GFM (GitHub Flavored Markdown) con tablas, checklists, resaltado, enlaces automáticos y emojis.

Ejemplo de checklist en un issue

```
- [x] Definir arquitectura
- [ ] Crear pruebas unitarias
- [ ] Redactar documentación final
```

15.3 Documentación en GitLab

•Muestra README.md como página principal del repositorio.

•Soporta CONTRIBUTING.md y CHANGELOG.md.

•Archivos .md dentro de /docs pueden formar sitios navegables (con MkDocs o Jekyll a través de GitLab Pages).

•Issues y Merge Requests aceptan Markdown con listas, código y tablas.

15.4 Documentación en Bitbucket

•README.md se muestra automáticamente en la página principal.

•Soporta documentación en carpetas /docs.
•Issues y wikis con soporte Markdown.
•Integración con herramientas Atlassian (por ejemplo, Confluence).
👉 A diferencia de GitHub y GitLab, Bitbucket es más usado en entornos empresariales, pero la base de documentación en .md es la misma.

15.5 Ventajas del uso de Markdown en documentación técnica

•Simplicidad: escribir documentación clara sin aprender un lenguaje complejo.
•Legibilidad en texto plano: el .md se entiende aun sin renderizarse.
•Control de versiones: se integra perfectamente con Git.
•Compatibilidad multiplataforma: funciona en GitHub, GitLab, Bitbucket, Obsidian, VS Code, etc.
•Extensibilidad: con GFM y otras variantes se añaden tablas, tareas, emojis.

15.6 Buenas prácticas

•Incluir siempre un README.md en proyectos.
•Organizar documentación extensa en /docs con varios .md.
•Usar índice con enlaces internos en documentos largos.
•Aprovechar listas de tareas (- [ ]) para seguimiento de pendientes.
•Incluir ejemplos de código con resaltado de sintaxis.
•Agregar imágenes o diagramas cuando aporten claridad.
•Mantener consistencia en títulos (#, ##, ###) para facilitar la navegación.

Conclusión

Markdown es hoy el estándar de documentación técnica en repositorios de software. Archivos como README.md, CONTRIBUTING.md y CHANGELOG.md son esenciales en GitHub, GitLab y Bitbucket. Su simplicidad, sumada a extensiones como GFM, lo hace ideal para manuales, guías, issues, wikis y documentación colaborativa.
¿Listo para practicar lo aprendido? Visita el visor de Markdown y pon en práctica los conceptos.

Markdown en blogs y gestores de contenido

Markdown se ha convertido en un estándar en la web no solo para documentación técnica, sino también para la creación de blogs y sitios de contenido. Gracias a su

simplicidad, legibilidad y portabilidad, muchos generadores de sitios estáticos y CMS lo adoptaron como formato principal.

16.1 Plataformas que lo usan

1. Jekyll

Uno de los primeros generadores de sitios estáticos (escrito en Ruby). Es el motor oficial detrás de GitHub Pages, por lo que es popular en proyectos de código abierto. Los posts se guardan como archivos `.md` en la carpeta `_posts/`.

Ejemplo de post en Jekyll:

```
---
layout: post
title: "Mi primer artículo en Jekyll"
date: 2025-09-15
---

# Bienvenidos

Este es un artículo escrito en **Markdown**.
```

👉 El bloque inicial `---` es YAML front matter, donde se definen metadatos (título, fecha, etiquetas, etc.).

2. Hugo

Generador de sitios estáticos escrito en Go, reconocido por su velocidad. Los artículos se escriben en `.md` dentro de la carpeta `content/`.

Ejemplo:

```
---
title: "Tutorial de Markdown en Hugo"
date: 2025-09-15
draft: false
---

Escribir en **Markdown** con Hugo es simple y rápido.
```

👉 Hugo admite front matter en YAML, TOML o JSON.

3. Gatsby

Framework basado en React que combina contenido en Markdown con componentes dinámicos; usa GraphQL para obtener datos desde archivos `.md`.

4. Hexo

Generador de sitios estáticos escrito en Node.js y orientado a usuarios de JavaScript. Permite escribir artículos en `.md` y ofrece variedad de temas y plugins.

5. Otros generadores populares

•Pelican (Python)
•MkDocs (Python), especializado en documentación técnica
•Docusaurus (React), ideal para documentar productos
👉 En todos estos casos, Markdown es el formato principal para escribir páginas y artículos.

16.2 Integración con CMS

Los CMS tradicionales (como WordPress, Drupal, Joomla) usan editores visuales (WYSIWYG), pero el auge del contenido ligero y portable llevó a integrar Markdown de forma nativa o mediante plugins.

1. WordPress

De fábrica usa el editor visual Gutenberg, pero se puede integrar Markdown con plugins como Jetpack Markdown o WP Markdown Editor, para escribir en .md y renderizar a HTML automáticamente.

2. Ghost

CMS moderno y minimalista enfocado en blogs y newsletters; soporta Markdown de forma nativa como formato de edición principal.

3. Netlify CMS

Headless CMS para sitios estáticos que se integra con varios generadores; guarda artículos .md directamente en el repositorio Git.

4. Headless CMS modernos

Contentful, Sanity, Strapi, Forestry permiten gestionar contenido desde una interfaz web y almacenar posts en Markdown; muy usados con Next.js, Gatsby, Hugo.

5. Notion y Obsidian (semi-CMS)

Exportan notas en Markdown, lo que facilita migrarlas a generadores de sitios o publicarlas en la web.

16.3 Ventajas de usar Markdown en blogs y CMS

•Simplicidad: escribir sin preocuparse por etiquetas HTML.
•Portabilidad: un mismo artículo .md puede moverse entre plataformas.
•Rapidez: ideal para sitios estáticos que se publican automáticamente desde repositorios.

•Versionado: los archivos .md se controlan fácilmente con Git.
•Compatibilidad: funciona en GitHub Pages, Netlify, WordPress (plugins), Ghost, etc.

16.4 Buenas prácticas

•Usar front matter para definir título, fecha, autor y etiquetas.
•Organizar el contenido en carpetas (/content, /_posts, /docs).
•Mantener consistencia en nombres de archivos (YYYY-MM-DD-mi-post.md).
•Optimizar imágenes y guardarlas en /assets o /static.
•Usar enlaces relativos para facilitar migraciones.
•Probar el documento en el renderizador final (no todos los CMS soportan las mismas extensiones).

16.5 Ejemplo completo de post en Hugo

```
---
title: "Guía práctica de Markdown en blogs"
date: 2025-09-15
author: "Diego"
tags: ["markdown", "blogs", "tutorial"]
draft: false
---

# Guía práctica de Markdown en blogs

Markdown es un formato **ligero y potente** que permite crear
artículos sin distracciones.

## Ventajas
- Fácil de aprender
- Legible en texto plano
- Compatible con múltiples CMS y plataformas

## Ejemplo de código
```python
print("Hola desde un post en Markdown!")
```

Conclusión
- Markdown es ampliamente usado en **blogs y gestores de
contenido** por su **simplicidad y compatibilidad**.
- Generadores como **Jekyll, Hugo, Gatsby y Hexo** se basan en
`.md` para crear páginas y artículos.
- CMS modernos como **Ghost, Netlify CMS** y headless CMS como
Contentful o Strapi integran Markdown como formato estándar.
- Es una herramienta **universal, portable y eficiente** para
crear contenido web.
```

Conclusión

Markdown es el estándar de facto para blogs técnicos y gestores de contenido modernos. Al compilar con generadores como Hugo o Jekyll, los archivos `.md` se transforman en páginas listas para publicarse.

Conversión de Markdown a otros formatos

Una de las grandes ventajas de Markdown es su portabilidad. Al ser texto plano, puede transformarse fácilmente a múltiples formatos sin perder estructura: HTML, PDF, DOCX, ODT, EPUB y presentaciones (por ejemplo con Reveal.js).

17.1 Pandoc: la herramienta universal

Pandoc es considerado el "cuchillo suizo" de la conversión de documentos. Fue creado por John MacFarlane y soporta decenas de formatos de entrada y salida.

Instalación

Linux (Debian/Ubuntu):
`sudo apt install pandoc`
macOS (Homebrew):
`brew install pandoc`
Windows: descargar desde el instalador oficial del sitio de Pandoc.

Uso básico

```
pandoc archivo.md -o salida.pdf
pandoc archivo.md -o salida.docx
pandoc archivo.md -o salida.html
```

👉 Pandoc interpreta Markdown y lo convierte al formato deseado automáticamente.

17.2 Conversión a HTML

Markdown fue pensado como alternativa simple a HTML, por lo que la conversión es directa.

`pandoc articulo.md -o articulo.html`

Se pueden aplicar plantillas y CSS personalizados para dar estilo.

17.3 Conversión a PDF

Para exportar a PDF, Pandoc suele apoyarse en un motor adicional como LaTeX, wkhtmltopdf o WeasyPrint.

`pandoc informe.md -o informe.pdf`

Personalizar tamaño de papel y fuente:

```
pandoc informe.md -o informe.pdf \
  --variable fontsize=12pt \
  --variable papersize=a4
```

17.4 Conversión a DOCX (Microsoft Word)

```
pandoc notas.md -o notas.docx
```

El archivo resultante conserva títulos, listas, enlaces e imágenes, listo para editar en Word.

17.5 Otras herramientas de conversión

Además de Pandoc, existen muchas herramientas y editores que exportan Markdown:

•Typora: exporta a PDF, DOCX, HTML, LaTeX.

•Obsidian: permite exportar notas a PDF.

Plugins y librerías

•markdown-pdf (Node.js): convierte .md a PDF por CLI.

•grip (Python): renderiza un .md como lo hace GitHub.

•md-to-pdf (Node.js): usa Puppeteer para exportar a PDF con estilos CSS.

17.6 Ejemplo práctico con Pandoc

Supongamos un archivo tutorial.md:

```
# Tutorial de Markdown

Markdown es un lenguaje ligero para crear documentos.

## Ventajas
- Fácil de escribir
- Legible en texto plano
- Exportable a múltiples formatos
```

Exportar a varios formatos:

```
# A PDF
pandoc tutorial.md -o tutorial.pdf

# A DOCX
pandoc tutorial.md -o tutorial.docx

# A HTML
pandoc tutorial.md -o tutorial.html
```

👉 Resultado: el mismo documento en tres formatos distintos sin modificar el original.

17.7 Buenas prácticas al exportar

•Usar encabezados bien jerarquizados (#, ##, ###) para que se mapeen a títulos en DOCX o PDF.

•Mantener imágenes con rutas relativas (img/archivo.png) para incluirlas correctamente.
•Probar la exportación en varios visores (el estilo puede variar según el motor).
•Para PDFs profesionales, integrar Pandoc con LaTeX para mejor control tipográfico.
•Para HTML, aplicar una hoja de estilo CSS personalizada.

Conclusión

Markdown es extremadamente flexible y puede transformarse en PDF, DOCX, HTML, EPUB y más. Pandoc es la herramienta más completa y potente para estas conversiones, y existen alternativas ligeras como Typora, Mark Text, Obsidian y varias librerías en Node.js o Python.

Herramientas y editores para Markdown

Aunque Markdown puede escribirse en cualquier editor de texto plano (Notepad, nano, Vim), existen editores y entornos modernos que facilitan su uso con previsualización en tiempo real, resaltado, integración con CMS y exportación a otros formatos.
A continuación, un repaso por las herramientas más populares para trabajar con Markdown.

18.1 Visual Studio Code (VS Code)

Visual Studio Code es uno de los editores más utilizados gracias a su soporte nativo y extensiones para Markdown.
Funcionalidades destacadas
•Soporte nativo para archivos .md.
•Previsualización integrada: atajo Ctrl+Shift+V.
•Resaltado para tablas, listas y bloques de código.
•Exportación a PDF/HTML con extensiones.
Extensiones recomendadas
•Markdown All in One: atajos, autocompletado, TOC.
•Markdown Preview Enhanced: vista previa avanzada (Mermaid, PlantUML).
•Paste Image: pegar capturas en un .md.

18.2 Typora

Typora es un editor minimalista enfocado en escribir Markdown de manera fluida.
•Edición y previsualización integradas.
•Exportaciones a PDF, DOCX, HTML, LaTeX, EPUB.
•Tablas y diagramas; MathJax y Mermaid.

•Modo sin distracciones.

18.3 Obsidian

Obsidian es una app para gestión del conocimiento (PKM) usando Markdown como base.

•Notas interconectadas con [[enlaces]].

•Grafo de relaciones entre notas.

•Amplio ecosistema de plugins.

•Almacenamiento local en archivos .md.

18.4 Otras herramientas destacadas

•Mark Text: código abierto y multiplataforma.

•Zettlr: para académicos, referencias y LaTeX.

•Dillinger: editor web con exportación a nubes.

•HackMD / CodiMD: edición colaborativa en tiempo real.

•StackEdit: editor online con sincronización.

18.5 Extensiones y previsualizadores

Más allá de los editores, hay previsualizadores y extensiones útiles:

Previsualizadores

•grip (Python): renderiza como GitHub.

•Marked (macOS): vistas rápidas.

•markdown-preview (Atom): nativo en Atom (aunque Atom está discontinuado).

Extensiones de navegador

•Markdown Viewer: abrir .md directo en el navegador.

•Docsify: convierte un repo con .md en documentación navegable.

Plugins de exportación

•md-to-pdf (Node.js): exporta a PDF con CSS.

•pandoc: herramienta universal de conversión.

18.6 Buenas prácticas al elegir editor

•Si programas: VS Code + Markdown All in One.

•Si escribes: Typora o Mark Text.

•Si gestionas conocimiento: Obsidian.

•Si necesitas colaboración online: HackMD o StackEdit.

•Si necesitas exportar a muchos formatos: usar Pandoc con tu editor.

18.7 Ejemplo de flujo de trabajo

1. Escribir en Obsidian para organizar ideas.

2.Editar en VS Code con extensiones de tablas y TOC.
3.Exportar versión final en Typora a PDF y DOCX.
4.Publicar el .md en GitHub o en un blog con Jekyll/Hugo.

Conclusión

Markdown puede escribirse en cualquier editor, pero herramientas como VS Code, Typora y Obsidian hacen el trabajo más productivo. Previsualizadores y extensiones aportan valor para exportar, colaborar y automatizar. La elección depende del objetivo: código, escritura, gestión de conocimiento o colaboración.

Buenas prácticas al escribir en Markdown

Markdown es simple y flexible, pero esa misma libertad puede llevar a documentos inconsistentes o difíciles de leer si no se aplican reglas claras de estilo. Las buenas prácticas aseguran que el texto sea legible en texto plano y consistente cuando se renderiza en distintas plataformas (GitHub, GitLab, CMS, editores de notas, etc.).

19.1 Legibilidad

La legibilidad es la capacidad de un documento para ser comprendido de manera rápida, tanto en texto plano como en su versión renderizada.

Reglas clave

Usar encabezados claros y jerárquicos

Evitar títulos genéricos como “Tema 1”; mejor “1. Introducción a Markdown”.

```
# Curso de Markdown
## Tema 1: Introducción
### Subtema: Historia
```

Separar secciones con líneas en blanco — Aumenta la claridad del texto en bruto.

Incorrecto:

```
## Instalación
Paso 1: descargar.
Paso 2: instalar.
```

Correcto:

```
## Instalación

Paso 1: descargar.
Paso 2: instalar.
```

Evitar párrafos muy largos — Mejor dividir en bloques cortos.

Usar listas para enumerar ideas — Más claras que párrafos corridos.

```
- Fácil de usar
- Portable
- Compatible con múltiples plataformas
```

Mantener texto alternativo en imágenes — Facilita accesibilidad.

```
![Logo de Markdown](logo.png)
```

19.2 Consistencia en estilos y formatos

La consistencia hace que el documento luzca profesional y homogéneo.

Reglas clave

Escoger un estilo de listas y mantenerlo

Incorrecto (mezcla de - y *):

```
- Manzana
* Banana
- Naranja
```

Correcto:

```
- Manzana
- Banana
- Naranja
```

Usar el mismo estilo para énfasis — Markdown admite `*cursiva*` y `_cursiva_`, pero no conviene mezclarlos. Recomendación: elegir `*` para cursiva y `**` para negrita.

Encabezados: un solo # por nivel — No duplicar `##` y `###` en el mismo título.

Incorrecto:

```
### ### Instalación
```

Correcto:

```
## Instalación
```

Nombrar enlaces de manera descriptiva — Evitar “clic aquí”.

```
[Documentación oficial de Python](https://www.python.org)
```

Formatear bloques de código correctamente — Usar triple backticks y especificar lenguaje.

```
def saludo():
    print("Hola, Markdown!")
```

Mantener consistencia en tablas — Alinear pipes para legibilidad.

```
| Lenguaje    | Popularidad |
|------------|-------------|
| Python      | Alta        |
| JavaScript | Muy alta    |
```

19.3 Ejemplo de buenas prácticas

```
# Guía de Markdown

Markdown es un lenguaje de marcado ligero para crear documentos.

## Características principales
- Legible en texto plano
- Fácil de convertir a otros formatos
- Compatible con múltiples plataformas
```

```
## Ejemplo de código
```javascript
function sumar(a, b) {
  return a + b;
}
console.log(sumar(3, 4));
```

Recursos adicionales
- [Guía oficial](https://daringfireball.net/projects/markdown/)
- [GitHub Flavored Markdown](https://github.github.com/gfm/)
```

👉 Este ejemplo cumple con: encabezados jerárquicos, listas consistentes, código con resaltado y enlaces descriptivos.

19.4 Beneficios de aplicar buenas prácticas

•📖 Mayor legibilidad — en texto plano y renderizado.
•🎨 Documentos consistentes y profesionales.
•🤝 Colaboración más fácil — lectura y edición sin confusiones.
•🔄 Facilidad de conversión — a PDF, HTML o DOCX.
•♿ Accesibilidad — mejor para lectores de pantalla.

Conclusión

La legibilidad se logra con párrafos cortos, listas claras, imágenes con texto alternativo y buena separación visual. La consistencia en estilos mantiene profesional el documento: mismo estilo de listas, negritas/cursivas coherentes, enlaces descriptivos y encabezados bien jerarquizados. Un documento Markdown con buenas prácticas es más fácil de mantener, compartir y convertir a otros formatos.

100 plataformas donde se usa Markdown

1. Repositorios y plataformas de desarrollo

•GitHub — README.md, issues, PRs y wikis (GFM).
•GitLab — README, issues, merge requests y wikis.
•Bitbucket — README y wikis de proyecto.
•SourceForge — descripciones y documentación de proyectos.
•Gitea — README, issues y wikis.
•Gogs — documentación y README de repos.
•Azure DevOps (Repos) — wikis y descripciones con Markdown.
•AWS CodeCommit — README y comentarios en revisiones.
•Google Cloud Source Repositories — README y docs en repos.

•Phabricator — tareas y wikis (Remarkup/Markdown).

2. Documentación y wikis

•GitHub Wiki — páginas de wiki en Markdown.
•GitLab Wiki — wikis con Markdown.
•Bitbucket Wiki — documentación en Markdown.
•Docusaurus — genera sitios de docs desde .md.
•MkDocs — sitios de documentación con Markdown.
•Read the Docs — publica docs a partir de .md/.rst.
•Slate — documentación de APIs en Markdown.
•Sphinx — admite Markdown vía extensión.
•Docsify — docs SPA desde archivos .md.
•Hugo Docs — documentación oficial en Markdown.

3. Gestores de contenido (CMS estáticos y dinámicos)

•Jekyll — posts y páginas en .md.
•Hugo — contenido y docs en .md.
•Gatsby — contenido Markdown vía plugins/MDX.
•Hexo — blogs escritos en Markdown.
•Pelican — artículos .md.
•Eleventy (11ty) — plantillas y contenido .md.
•Middleman — blogs/documentación con .md.
•Next.js (con MDX) — páginas .mdx.
•Nuxt Content (Vue) — contenido en Markdown.
•Metalsmith — genera sitios desde .md.

4. Blogs y plataformas de publicación

•Dev.to — artículos en Markdown.
•Hashnode — posts con Markdown.
•Ghost — editor con soporte Markdown.
•WordPress (plugins Markdown) — redactar entradas en Markdown.
•Medium (importadores) — importar desde Markdown.
•Write.as — blogging con Markdown.
•Bear Blog — blogs en Markdown.
•Blot.im — blog desde archivos .md.
•Netlify CMS — contenido guardado en .md.
•Forestry — CMS para contenido Markdown.

5. Aplicaciones de notas y productividad

•Obsidian — notas enlazadas en .md.
•Notion — importa/exporta Markdown.
•Typora — edición Markdown con vista unificada.
•Mark Text — editor Markdown multiplataforma.
•Zettlr — escritura académica en Markdown.
•Joplin — notas y sincronización en .md.
•Boost Note — notas para devs con Markdown.
•QOwnNotes — notas locales en Markdown.
•Simplenote — soporte básico de Markdown.
•Bear App — notas con sintaxis Markdown.

6. Foros y comunidades

•Reddit — formato tipo Markdown en posts/comentarios.
•Stack Overflow — Markdown en preguntas y respuestas.
•Stack Exchange — Markdown en toda la red.
•Discourse — foros modernos con Markdown.
•Forem — motor de comunidades con Markdown.
•Spectrum.chat — hilos con Markdown.
•Hashnode discussions — debates en Markdown.
•DEV Community — artículos Markdown.
•Indie Hackers — posts con Markdown.
•Lobsters — comentarios con Markdown.

7. Chats y colaboración

•Slack — soporte parcial de Markdown en mensajes.
•Discord — formato inspirado en Markdown.
•Mattermost — mensajes y posts en Markdown.
•Rocket.Chat — soporte de Markdown en chat.
•Zulip — Markdown en hilos y mensajes.
•Twist — formato Markdown en conversaciones.
•Keybase — Markdown en chats seguros.
•Matrix / Element — mensajes con Markdown.
•Gitter — Markdown en salas de devs.
•Telegram (MarkdownV2 en bots) — formato en mensajes de bots.

8. Gestores de proyectos

•Trello — descripciones y comentarios con Markdown.
•Jira — sintaxis Markdown-like en issues.
•Asana — formato tipo Markdown en tareas.
•Basecamp — mensajes y docs con Markdown.
•Shortcut (antes Clubhouse) — historias/comentarios en Markdown.
•Taiga — descripciones de user stories en Markdown.
•YouTrack — Markdown en descripciones y comentarios.
•Kanboard — Markdown en tarjetas.
•Redmine — soporte vía plugin Markdown.
•Monday.com — soporte parcial de Markdown.

9. Plataformas de e-learning y escritura académica

•Coursera — notebooks/foros con Markdown.
•EdX — contenido técnico en Markdown.
•Moodle — plugins de Markdown en ediciones.
•Canvas LMS — páginas con formato Markdown.
•Overleaf — importación/convertir desde Markdown.
•RStudio (RMarkdown) — informes reproducibles.
•Jupyter Notebooks — celdas de Markdown.
•Kaggle Notebooks — celdas de Markdown.
•Google Colab — celdas de Markdown.
•Binder — notebooks ejecutables con Markdown.

10. Otros usos diversos

•GitHub Pages — publica sitios desde Markdown.
•Netlify — despliegue de sitios basados en .md.
•Vercel — Next.js/MDX y contenido Markdown.
•Readme.com — documentación en Markdown.
•Apiary — documentación de API en Markdown.
•SwaggerHub — descripciones con soporte Markdown.
•Postman Docs — docs de API con Markdown.
•npm — README.md de paquetes.
•PyPI — descripción del paquete en Markdown.
•Docker Hub — README.md para imágenes.

11. Plataformas de chat e IA conversacional

•ChatGPT (OpenAI) — respuestas en Markdown (títulos, listas, código).
•Claude (Anthropic) — salidas estructuradas en Markdown.
•Perplexity AI — encabezados, viñetas y enlaces en Markdown.
•Google Gemini — código en bloques Markdown y enlaces.
•GitHub Copilot — sugerencias y docs en Markdown en IDEs.
•Codeium — snippets y ejemplos en Markdown.
•Tabnine — código y documentación en Markdown.
•Phind — resultados estructurados en Markdown.
•Replit Ghostwriter — código y docs en Markdown en el editor.
•Cursor AI — respuestas y código en Markdown.